

## Lab 9

### Controlling the Spider Robot with C++

**Name:** Ethan Greenhouse

greenhouse.e@northeastern.edu

**Name:** Gavin Lee

lee.gavin@northeastern.edu

Date: December 4, 2025

Due Date: December 7, 2025

### **Abstract**

This lab worked with a hexapod “Spider” robot. The spider system has three classes: the Spider, SpiderLeg, and ServoMotor, and they all coordinate together to initialize and move the robot. A provided program is given that is able to successfully compile and do forward walking. It was very jittery during initialization, and some legs hyperextended.

This lab modifies the codebase in order to successfully move backward, turn right, and turn left by turning the servos and raising the legs with correct timing. The not fully functioning initialization process was made less jittery and worked correctly by fixing the code. Each of these fixes used C++ to modify the code, and then the main() command interface also had to be changed and recompiled. The resulting system moved the spider in all directions and allowed it to smoothly initialize.

### **Introduction**

In this lab, the Spider robot demonstrates using functions and classes to operate hardware. C++ code is edited such that it controls the ServoMotor, SpiderLeg, and Spider functions. The original program initialized the robot, tested each leg, and has code for a forward walking function of the tripods. The initialization would fail and hyper extend while also jittering. The goals of this lab were to look through and understand the given Spider code. The initialization was improved so that it would go through it smoothly. Next, a backward move, left, and right turning functions were implemented. Then the Spider was tested and verified that everything was working correctly.

## **Setup**

### Hardware

- Spider Robot
- Ethernet and power cable

### Software

- MobaXterm

### Files Used

- Given Spider.zip
- Our coded C++ program

## **Design**

### **Implement Backward Walking**

Backward walking was done by reversing the forward tripod gait but keeping the same logic. The robot still alternates between tripods, but the hip joint direction has changed. The code determines which tripod should step next based on the previous direction. It takes the last tripod used and raises that tripod, rotates its hip joints backward, rotates the opposite tripod forward, and lowers the lifted tripod. The code below shows the hip direction needed for moving backward. Code for the full function is shown in Appendix i.

```

// Backward walking
// Reverse of forward: hips rotate opposite direction, Tripod 1
// swings backward while Tripod 2 pushes forward

void MoveBackward() {
    if ((lastStep == TRIPOD2 && lastDir == BACK) || (lastStep
== TRIPOD1 && lastDir == FWD)){
        // Lift TRIPOD1 legs
        MoveTripod(TRIPOD1, SpiderLeg::Knee, Knee_Up_Base,
Knee_Up_Base, Knee_Up_Base);
        WaitReady();
        // Move hips in opposite direction from forward
        MoveTripod(TRIPOD1, SpiderLeg::Hip, HipF_Base - 20,
HipM_Base - 20, HipB_Base - 20);
        MoveTripod(TRIPOD2, SpiderLeg::Hip, HipF_Base + 20,
HipM_Base + 20, HipB_Base + 20);
        WaitReady();
        // Lower TRIPOD1 legs
        MoveTripod(TRIPOD1, SpiderLeg::Knee, Knee_Down_Base,
Knee_Down_Base, Knee_Down_Base);
        WaitReady();
        lastStep = TRIPOD1;
    }
}

```

### Implement Turning

Turning requires a multiple-step process of hip motions in order to rotate the robot's body around its centre. The first step to rotating right (CW) involves raising legs in tripod B, rotating them clockwise, then lowering again. The second step involves raising legs in tripod R, rotating tripod B counterclockwise, then lowering again. This results in the spider staying in the same position, but turning slightly to the right. A similar but mirrored pattern rotates the robot left (CCW). Each turn had a separate function `RightTurn()` and `LeftTurn()`. The code snippet shows the main logic that creates the CCW turning. Code for the full function is shown in Appendix ii.

```

    // Tripod B rotates CCW: right side forward, Left side backward
    m_szLeg[LEG_RF]->MoveJoint(SpiderLeg::Hip, HipF_Base + 20); //
    Right front forward
    m_szLeg[LEG_LM]->MoveJoint(SpiderLeg::Hip, HipM_Base - 20); //
    Left middle backward
    m_szLeg[LEG_RB]->MoveJoint(SpiderLeg::Hip, HipB_Base + 20); //
    Right back forward
    WaitReady();

```

### Fix Initialization

The original initialization caused jittering servos, sudden leg hyperextension, and unstable startup posture. A couple of changes were required to fix the initialization sequence. The Init() function was used to set all hips, knees, and ankles to 0 degrees so no leg starts in an extended or rotated position. Then, the Standup() function uses a controlled, multi-stage process to set the spider to a standing state. The process does the following: hips are aligned to base angles, legs extend horizontally, and knees are lowered from 90 degrees to 45 degrees in 5-degree increments. The following changes are shown below. Code for the full Standup() function is shown in Appendix iii.

```

void Init()
{
    // Set all servos simultaneously to neutral position (0
    degrees) first
    for (int i = 0; i < LEG_NUM; i++){
        m_szLeg[i]->MoveJoint(SpiderLeg::Hip, 0.0);
        m_szLeg[i]->MoveJoint(SpiderLeg::Knee, 0.0);
        m_szLeg[i]->MoveJoint(SpiderLeg::Ankle, 0.0);
    }
    WaitReady();

    // Small delay to ensure all servos have initialized
    usleep(500000); // 0.5s delay
}

```

```
// Step 3: Gradually Lower the body by bending knees  
// Move slowly from 90 degrees down to 45 degrees  
float KneeAngle = 90;  
const float AnkleAngle = 45.0;  
while (bSuccess && KneeAngle >= Knee_Down_Base)  
{  
    for (int i = 0; i < LEG_NUM; i++)  
    {  
        m_szLeg[i]->MoveJoint(SpiderLeg::Knee, KneeAngle);  
        m_szLeg[i]->MoveJoint(SpiderLeg::Ankle, AnkleAngle);  
    }  
    bSuccess = WaitReady();  
    KneeAngle -= 5.0;  
}
```

## **Results and Analysis**

The modified spider code compiled and executed successfully without errors. The robot's movements and initialization were an improvement from the original, with the jitter and hyperextension observed at the beginning fixed. The robot steadily and steadily went into a standing position without jitter. All six legs move in sync in order to do this. After being in a standing position, each leg takes a turn raising, then they go up in pairs. This confirmed the symmetry and synchronization of the SpiderLeg objects.

The backward function worked as intended when the command 'b' was entered. The tripod groups reversed the swing and stance phases. This causes the robot to step backward one stride. It moved smoothly and without failure.

The turning of the spider was also tested and worked correctly. Typing the command 'l' produced a left rotation. The two tripods worked in sync as described in the lecture slides, which caused the Spider to turn left. The turning motion was smooth, and there were no leg collisions. The same thing happened for the right turning command 'r', but everything was reversed, such that the robot turned to the right.

The servo coordination was used with the ServoMotor moving the SpiderLeg, which, in the end, correctly moved the Spider. Each method executed responded to our commands which showed that the C++ code and control stack compiled correctly.

## **Conclusion**

This lab showed how to fully go through the process of a multi-layered software controlling a somewhat complex robotic system using C++. The provided codebase was used as a structure and was modified at the class and method levels. This allowed the robot to correctly move backwards, turn left, and turn right. It also allowed the spider to initialize as expected without jitter and without hyperextensions. The improvements used an understanding of servo control and object-oriented design. It also used the tripods' mechanics to get the Spider to function.

## **References**

[1] "Lab 9, Controlling the Spider Robot with C++," University, 4 December 2025.

## **Appendix**

Appendix i: MoveBackward() function

```
void MoveBackward() {
    if ((lastStep == TRIPOD2 && lastDir == BACK) || (lastStep
== TRIPOD1 && lastDir == FWD)){
        // Lift TRIPOD1 Legs
        MoveTripod(TRIPOD1, SpiderLeg::Knee, Knee_Up_Base,
Knee_Up_Base, Knee_Up_Base);
        WaitReady();
        // Move hips in opposite direction from forward
        MoveTripod(TRIPOD1, SpiderLeg::Hip, HipF_Base - 20,
HipM_Base - 20, HipB_Base - 20);
        MoveTripod(TRIPOD2, SpiderLeg::Hip, HipF_Base + 20,
HipM_Base + 20, HipB_Base + 20);
        WaitReady();
        // Lower TRIPOD1 Legs
```

```

        MoveTripod(TRIPOD1, SpiderLeg::Knee, Knee_Down_Base,
Knee_Down_Base, Knee_Down_Base);
        WaitReady();
        lastStep = TRIPOD1;
    } else {
        // Lift TRIPOD2 Legs
        MoveTripod(TRIPOD2, SpiderLeg::Knee, Knee_Up_Base,
Knee_Up_Base, Knee_Up_Base);
        WaitReady();
        // Move hips in opposite direction from forward
        MoveTripod(TRIPOD1, SpiderLeg::Hip, HipF_Base + 20,
HipM_Base + 20, HipB_Base + 20);
        MoveTripod(TRIPOD2, SpiderLeg::Hip, HipF_Base - 20,
HipM_Base - 20, HipB_Base - 20);
        WaitReady();
        // Lower TRIPOD2 Legs
        MoveTripod(TRIPOD2, SpiderLeg::Knee, Knee_Down_Base,
Knee_Down_Base, Knee_Down_Base);
        WaitReady();
        lastStep = TRIPOD2;
    }
    lastDir = BACK;
}

```

#### Appendix ii: LeftTurn() function

```

void LeftTurn()
{
    // Complete Left Turn (CCW)

    // Phase 1 & 2: Raise Tripod B and rotate CCW
    MoveTripod(TRIPOD1, SpiderLeg::Knee, Knee_Up_Base, Knee_Up_Base,
Knee_Up_Base);
    WaitReady();

    // Tripod B rotates CCW: right side forward, left side backward
    m_szLeg[LEG_RF]->MoveJoint(SpiderLeg::Hip, HipF_Base + 20); //
Right front forward
    m_szLeg[LEG_LM]->MoveJoint(SpiderLeg::Hip, HipM_Base - 20); //

```

```

    Left middle backward
    m_szLeg[LEG_RB]->MoveJoint(SpiderLeg::Hip, HipB_Base + 20); //
    Right back forward
    WaitReady();

    // Phase 3: Lower Tripod B
    MoveTripod(TRIPOD1, SpiderLeg::Knee, Knee_Down_Base,
    Knee_Down_Base, Knee_Down_Base);
    WaitReady();

    // Phase 4: Raise Tripod R
    MoveTripod(TRIPOD2, SpiderLeg::Knee, Knee_Up_Base, Knee_Up_Base,
    Knee_Up_Base);
    WaitReady();

    // Phase 5: Tripod B ALL legs rotate CW (all back to base
    position)
    m_szLeg[LEG_RF]->MoveJoint(SpiderLeg::Hip, HipF_Base);
    m_szLeg[LEG_LM]->MoveJoint(SpiderLeg::Hip, HipM_Base);
    m_szLeg[LEG_RB]->MoveJoint(SpiderLeg::Hip, HipB_Base);
    WaitReady();

    // Phase 6: Lower Tripod R
    MoveTripod(TRIPOD2, SpiderLeg::Knee, Knee_Down_Base,
    Knee_Down_Base, Knee_Down_Base);
    WaitReady();

    lastStep = TRIPOD2;
    lastDir = FWD;
}

```

#### Appendix iii: Standup() function

```

void Standup()
{
    bool bSuccess;

    // Step 1: Position hips to base angles
    float fszJoin0Angle[] = {HipF_Base, HipM_Base, HipB_Base,

```



```

        HipF_Base, HipM_Base, HipB_Base};
    for (int i = 0; i < LEG_NUM; i++)
        m_szLeg[i]->MoveJoint(SpiderLeg::Hip, fszJoin0Angle[i]);

    bSuccess = WaitReady();

    // Step 2: Extend Legs horizontally first (Knee at 90, Ankle
at 45)
    for (int i = 0; i < LEG_NUM; i++)
    {
        m_szLeg[i]->MoveJoint(SpiderLeg::Knee, 90);
        m_szLeg[i]->MoveJoint(SpiderLeg::Ankle, 45);
    }
    bSuccess = WaitReady();

    // Step 3: Slowly Lower the body by bending knees from 90
degrees down to 45 degrees
    float KneeAngle = 90;
    const float AnkleAngle = 45.0;
    while (bSuccess && KneeAngle >= Knee_Down_Base)
    {
        for (int i = 0; i < LEG_NUM; i++)
        {
            m_szLeg[i]->MoveJoint(SpiderLeg::Knee, KneeAngle);
            m_szLeg[i]->MoveJoint(SpiderLeg::Ankle, AnkleAngle);
        }
        bSuccess = WaitReady();
        KneeAngle -= 5.0;
    }

    // Step 4: Test each leg by lifting and lowering one at a
time
    if (bSuccess) {
        std::cout << "Testing legs..." << std::endl;

        for (int i = 0; i < LEG_NUM; i++)
        {
            // Lift leg
            m_szLeg[i]->MoveJoint(SpiderLeg::Knee, Knee_Up_Base);

```

```
        WaitReady();

        // Lower Leg
        m_szLeg[i]->MoveJoint(SpiderLeg::Knee,
Knee_Down_Base);
        WaitReady();
    }

    std::cout << "Leg test complete!" << std::endl;
}

// Step 5: Final reset on success
if (bSuccess) Reset();
}
```